Lab 09 – Recursion

Task01:

Code:

```
/* ---Ascending order--- */

#include<iostream>
#include <utility>
#include <vector>
using namespace std;

/**
 * 1. Merge Sort
 */

void merge(vector<int>& arr, int st, int mid, int end){
    int i = st; // pointer on first array
    int j = mid+1; // pointer on second array
    vector<int> temp;

    while(i <= mid && j <= end){
        if(arr[i] < arr[j]){
            temp.push_back(arr[i]);
            i++;
        } else {
            temp.push_back(arr[j]);
            j++;
        }
    }

    // emptying the remaining
    while(i <= mid) temp.push_back(arr[i++]);
    while(j <= end) temp.push_back(arr[j++]);

    // now copying elements in the original array
    for(int a = st; a <= end; a++) arr[a] = temp[a-st];
}

void mergeSort(vector<int>& arr, int st, int end){
    if(st < end){ // means array is valid
        int mid = st+((end-st)/2);

        mergeSort(arr, st, mid);
        mergeSort(arr, mid+1, end);
```

```
        merge(arr, st, mid, end);
    }
}

/**
 * 2. Quick Sort
 */
int partition(vector<int>& arr, int st, int end){
    int pivot = arr[st]; // consdering first element as pivot, now will find
place for it
    int i = st+1, j = end;

    while(i < j){
        // making i to it's write position
        while(arr[i] <= pivot) i++; // (we'll through ith on the right of pivot)

        // making j
        while(arr[j] > pivot) j--;

        if(i <  j){ // they didn't cross each other
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[st], arr[j]);
    return j;
}

void quickSort(vector<int>& arr, int st, int end){
    if(st < end){
        int pivdx = partition(arr, st, end);

        quickSort(arr, st, pivdx-1);
        quickSort(arr, pivdx+1, end);
    }
}

int main(){

    cout << "Hello World!\n";


}
```

Task 02:

Code:

```
/* ---Descending order--- */

#include<iostream>
#include <vector>
using namespace std;


/* ***Merge Sort*** */
void merge(vector<int>& arr, int st, int mid, int end){
    int i = st, j = mid+1;
    vector<int> temp;

    while(i <= mid && j <= end){
        if(arr[i] > arr[j]){
            temp.push_back(arr[i]);
            i++;
        } else {
            temp.push_back(arr[j]);
            j++;
        }
    }

    while(i <= mid) temp.push_back(arr[i++]);
    while(j <= end) temp.push_back(arr[j++]);

    for(int a = st; a <= end; a++) arr[a] = temp[a-st];
}

void mergeSort(vector<int>& arr, int st, int end){
    if(st < end){
        int mid = st+((end-st)/2);

        mergeSort(arr, st, mid);
        mergeSort(arr, mid+1, end);

        merge(arr, st, mid, end);
    }
}

/* ***Quick Sort*** */
int partition(vector<int>& arr, int st, int end){
    int pivot = arr[st];
    int i = st+1, j = end;
```

```
    while(i < j){
        while(arr[i] >= pivot) i++; // keep moving, until you find less element
than pivot

        while(arr[j] < pivot) j--;

        if(i < j) swap(arr[i], arr[j]);
    }
    swap(arr[st], arr[j]);

    return j;
}

void quickSort(vector<int>& arr, int st, int end){
    if(st < end){
        int pivdx = partition(arr, st, end);

        quickSort(arr, st, pivdx-1);
        quickSort(arr, pivdx+1, end);
    }
}

int main(){

    cout << "hello World!\n";

}
```